

AI505
Optimization

Discrete Optimization
Dynamic Programming & Constraint Programming

Marco Chiarandini

Department of Mathematics & Computer Science
University of Southern Denmark

1. Dynamic Programming

2. Constraint Programming

1. Dynamic Programming

2. Constraint Programming

- Applied to problems with **optimal substructure** and **overlapping subproblems**
- **Optimal substructure** means an optimal solution can be constructed from optimal solutions to its subproblems
- **Overlapping subproblems** means solving each subproblem separately requires repeating certain operations
- **Dynamic programming** begins with desired problem and recurses down to smaller and smaller subproblems, retrieving the value of previously solved problems as necessary

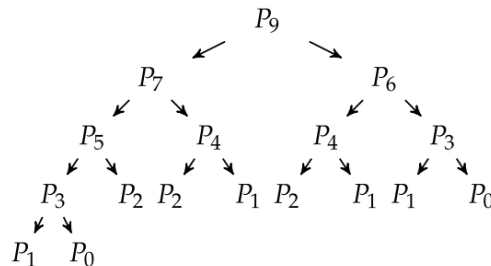
Overlapping Subproblems

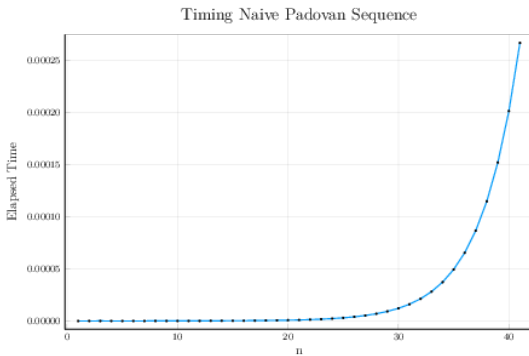
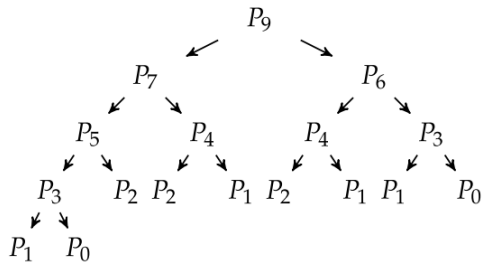
Example of Overlapping Subproblems: Padovan Sequence

$$P_n = P_{n-2} + P_{n-3}$$

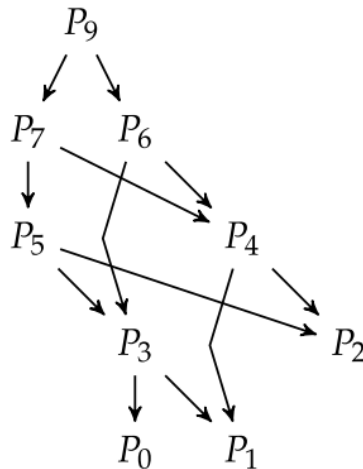
$$P_0 = P_1 = P_2 = 1$$

```
def padovan_naive(n, P=None):  
    if n < 3:  
        return 1  
    else:  
        return padovan_naive(n - 2, P) + ↪  
                ↪ padovan_naive(n - 3, P)
```





```
def padovan_topdown(n, P=None):  
    if P is None:  
        P = {}  
    if n not in P:  
        if n < 3:  
            P[n] = 1  
        else:  
            P[n] = padovan_topdown(n - 2, P) + ↪  
                ↪padovan_topdown(n - 3, P)  
    return P[n]  
  
def padovan_bottomup(n):  
    P = {0: 1, 1: 1, 2: 1}  
    for i in range(3, n + 1):  
        P[i] = P[i - 2] + P[i - 3]  
    return P[n]
```



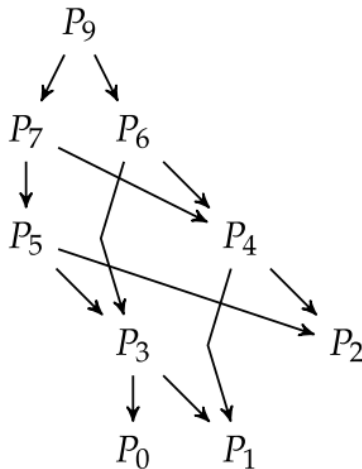
```

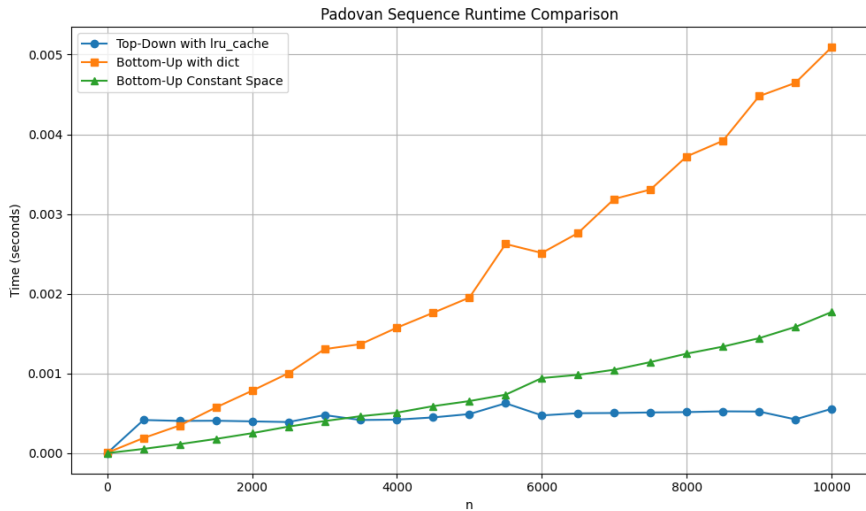
from functools import lru_cache

@lru_cache(maxsize=None)
def padovan_topdown(n):
    if n < 3:
        return 1
    return padovan_topdown(n - 2) + ↪
        ↪padovan_topdown(n - 3)

def padovan_bottomup_const(n):
    if n < 3:
        return 1
    p0, p1, p2 = 1, 1, 1 # Corresponds to P[0], P↪
        ↪[1], P[2]
    for _ in range(3, n + 1):
        p_next = p0 + p1
        p0, p1, p2 = p1, p2, p_next
    return p2

```





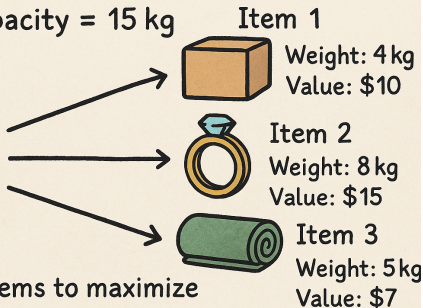
Optimal substructure

- **Principle of Optimality** (known as Bellman Optimality Conditions): Suppose that the solution of a problem is the result of a sequence of n decisions $D_1, D_2, \dots, D_n$; if a given sequence is optimal, then the first k decisions must be optimal, but also the last $n - k$ decisions must be optimal
- DP breaks down the problem into stages, at which decisions take place, and find a recurrence relation that relates each stage with the previous one

Example: Knapsack Problem

KNAPSACK PROBLEM

Capacity = 15 kg



Decision: pick items to maximize total value, but total weight must be ≤ 15 kg

- Trying to pack some number of items into a backpack
- Limited space in the backpack
- Each item has a specified value and size
- What is the best subset of items to include?

Recall: A MILP Formulation

$$\begin{aligned} & \underset{x}{\text{minimize}} && - \sum_{i=1}^n v_i x_i \\ & \text{subject to} && \sum_{i=1}^n w_i x_i \leq W \\ & && x_i \in \{0, 1\} \quad \text{for } i = 1, 2, \dots, n \end{aligned}$$

- Let the $V(i, w)$ (the so-called **value function**) be the maximum value achievable using the first i items and a knapsack capacity w .
- Consider the i th item. You can either use it or not.
 - If you don't use it, then the value of your knapsack will be

$$V(i - 1, w)$$

- If you use it, then the value of your knapsack will be

$$V(i - 1, w - w_i) + v_i$$

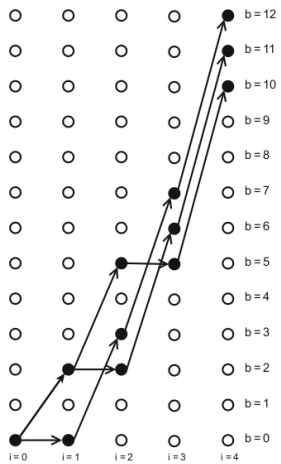
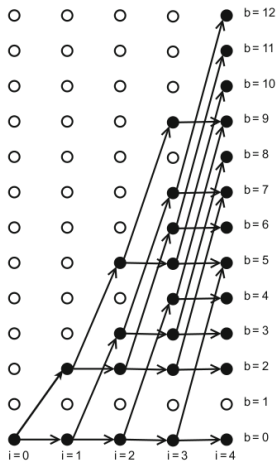
- For each item i and weight w :

$$V(i, w) = \begin{cases} 0 & \text{if } i = 0 \\ \max \begin{cases} V(i-1, w) & \text{(discard new item)} \\ V(i-1, w - w_i) + v_i & \text{(include new item)} \end{cases} & \text{if } w_i \leq w \\ V(i-1, w) & \text{if } w_i > w \end{cases}$$

- Optimal solution:

$$z^* = V(n, W)$$

and trace back to find the items collected.



Example

Capacity = 5 kg

Item	Weight	Value
1	1 kg	\$1
2	3 kg	\$4
3	4 kg	\$5

Step 1: Initialize

Create a table with (number of items + 1) rows and (capacity + 1) columns, filled with zeros.

```
w = 0 1 2 3 4 5
i=0 | 0 0 0 0 0 0
i=1 | 0
i=2 | 0
i=3 | 0
```

Step 2: Fill it step by step

```
Item 1 (w_1=1, v_1=1):
w = 0 1 2 3 4 5
i=0 | 0 0 0 0 0 0
i=1 | 0 1 1 1 1 1
i=2 | 0
i=3 | 0
```

Item 2 (w_2=3, v_2=4):

```
w = 0 1 2 3 4 5
i=0 | 0 0 0 0 0 0
i=1 | 0 1 1 1 1 1
i=2 | 0 1 1 4 5 5
i=3 | 0
```

Item 3 (w_3=4, v_3=5):

```
w = 0 1 2 3 4 5
i=0 | 0 0 0 0 0 0
i=1 | 0 1 1 1 1 1
i=2 | 0 1 1 4 5 5
i=3 | 0 1 1 4 5 6
```


Example: Traveling Salesman Problem

<https://www.math.uwaterloo.ca/tsp/>

The TSP asks for the shortest tour that starts from 0, visits all cities of the set $C = \{1, 2, \dots, n\}$ exactly once, and returns to 0, where the cost to travel from i to j is c_{ij} (with $(i, j) \in A$)

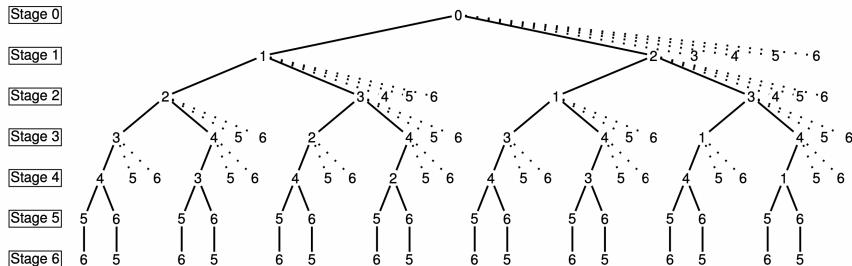
If the optimal solution of a TSP with six cities is $(0, 1, 3, 2, 4, 6, 5, 0)$, then...

- the optimal solution to visit $\{1, 2, 3, 4, 5, 6\}$ starting from 0 and ending at 5 is $(0, 1, 3, 2, 4, 6, 5)$
- the optimal solution to visit $\{1, 2, 3, 4, 6\}$ starting from 0 and ending at 6 is $(0, 1, 3, 2, 4, 6)$
- the optimal solution to visit $\{1, 2, 3, 4\}$ starting from 0 and ending at 4 is $(0, 1, 3, 2, 4)$
- the optimal solution to visit $\{1, 2, 3\}$ starting from 0 and ending at 2 is $(0, 1, 3, 2)$
- the optimal solution to visit $\{1, 3\}$ starting from 0 and ending at 3 is $(0, 1, 3)$
- the optimal solution to visit 1 starting from 0 is $(0, 1)$

~> The optimal solution is made up of a number of optimal solutions of smaller subproblems

Enumerate All Solutions of the TSP

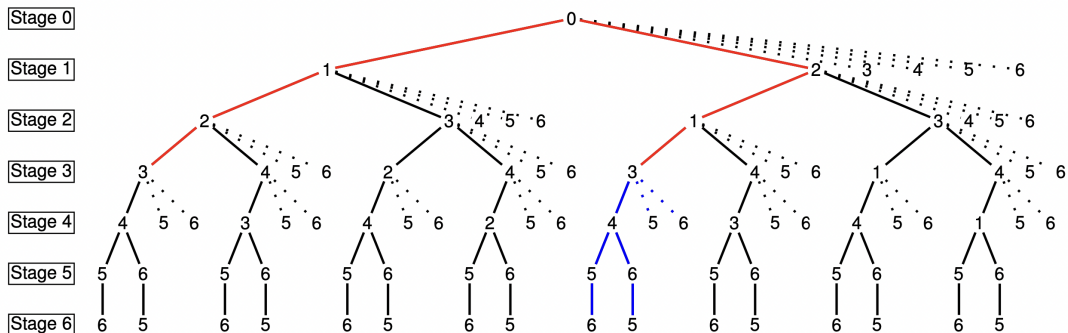
- A solution of a TSP with n cities derives from a sequence of n decisions, where the k th decision consists of choosing the k th city to visit in the tour



- The number of nodes (or states) grows exponentially with n
- At stage k , the number of states is $\binom{n}{k} k!$
- With $n = 6$, at stage $k = 6$, 720 states are necessary

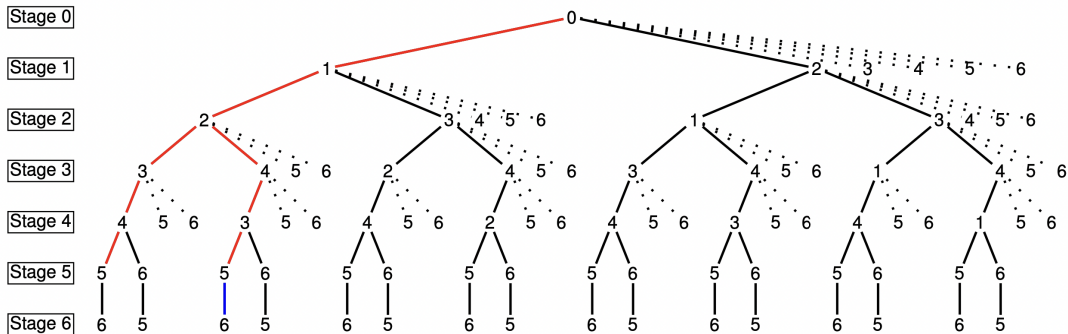
↪ DP finds the optimal solution by implicitly enumerating all states but actually generating only some of them

Are All States Necessary?



If path $(0, 1, 2, 3)$ costs less than $(0, 2, 1, 3)$, the optimal solution cannot be found in the blue part of the tree

Are All States Necessary?



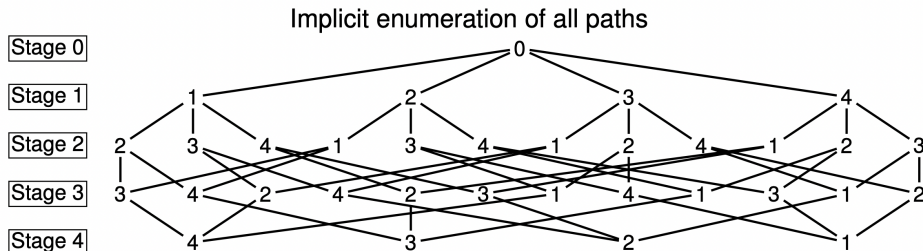
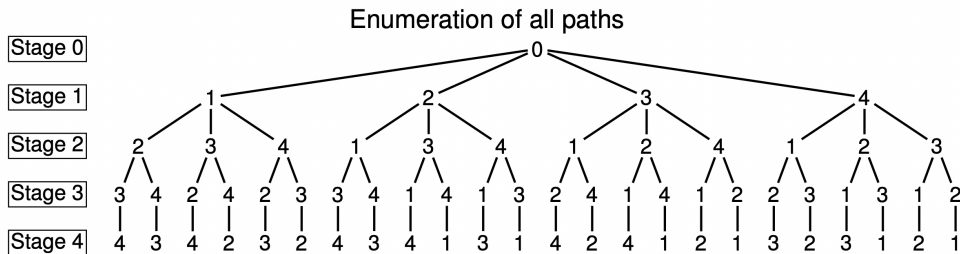
If path $(0, 1, 2, 3, 4, 5)$ costs less than $(0, 1, 2, 4, 3, 5)$, the optimal solution cannot be found in the blue part of the tree

Are All States Necessary?

- At stage k ($1 \leq k \leq n$), for each subset of cities $S \subseteq C$ of cardinality k , it is necessary to have only k states (one for each of the cities of the set S)
- At state $k = 3$, given the subset of cities $S = \{1, 2, 3\}$, three states are needed:
 - the shortest-path to visit S by starting from 0 and ending at 1
 - the shortest-path to visit S by starting from 0 and ending at 2
 - the shortest-path to visit S by starting from 0 and ending at 3
- At stage k , $\binom{n}{k} k$ states are required to compute the optimal solution (not $\binom{n}{k} k!$)

#States n = 6			
Stage	$\binom{n}{k} k!$	$\binom{n}{k} k$	
1	6	6	
2	30	30	
3	120	60	
4	360	60	
5	720	30	
6	720	6	

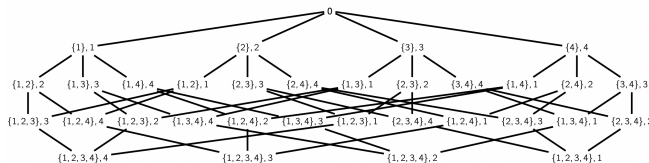
Dynamic Programming
Constraint Programming



Dynamic Programming Recursion for the TSP I

- Given a subset $S \subseteq C$ of cities and $k \in S$, let $f(S, k)$ be the optimal cost of starting from 0, visiting all cities in S , and ending at k
- Begin by finding $f(S, k)$ for $|S| = 1$, which is $f(\{k\}, k) = c_{0k}, \forall k \in C$
- To compute $f(S, k)$ for $|S| > 1$, the best way to visit all cities of S by starting from 0 and ending at k is to consider all $j \in S \setminus \{k\}$ immediately before k , and look up $f(S \setminus \{k\}, j)$, namely

$$f(S, k) = \min_{j \in S \setminus \{k\}} \{f(S \setminus \{k\}, j) + c_{jk}\}$$



- The optimal solution cost z^* of the TSP is $z^* = \min_{k \in C} \{f(C, k) + c_{k0}\}$

DP Recursion from [Held and Karp (1962)]

1. **Initialization.** Set $f(\{k\}, k) = c_{0k}$ for each $k \in C$

2. **RecursiveStep.** For each stage $r = 2, 3, \dots, n$, compute

$$f(S, k) = \min_{j \in S \setminus \{k\}} \{f(S \setminus \{k\}, j) + c_{jk}\} \forall S \subseteq C : |S| = r \text{ and } \forall k \in S$$

3. **Optimal Solution.** Find the optimal solution cost z^* as

$$z^* = \min_{k \in C} \{f(C, k) + c_{k0}\}$$

- With the DP recursion, TSP instances with up to 25 - 30 customers can be solved to optimality; other solution techniques (i.e., branch-and-cut) are able to solve TSP instances with up to... 85900 customers
- Nonetheless, DP recursions represents the state-of-the-art solution techniques to solve a wide variety of PDPs

**Thiago Serra** ✓ • 2nd
Assistant Professor of Business Analytics at University of Iowa
4d • 🌐

[+ Follow](#) ...

Bill Cook strikes again: shortest tour of 81,998 bars in South Korea is the top story tonight at Hacker News

Link: <https://lnkd.in/djSAkt5E>

📶 T-Mobile 📶

11:30 PM

🌙 🔒 🔋

Hacker News

Y

new | past | comments | ask | show | jobs | login

submit

1. ▲ Shortest-possible walking tour to 81,998 bars in South Korea (uwaterloo.ca)
150 points by geeknews 4 hours ago | hide | 32 comments

2. ▲ Show HN: My from-scratch OS kernel that runs DOOM (github.com/unmappedstack)
100 points by UnmappedStack 3 hours ago | hide | 19 comments

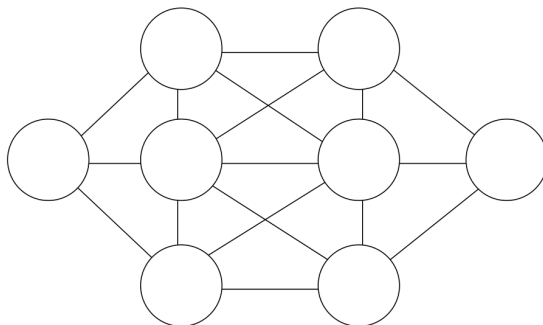
3. ▲ How a 20 year old bug in GTA San Andreas surfaced in Windows 11 24H2 (cookieplmonster.github.io)
962 points by vett 14 hours ago | hide |

- Dynamic programming is a powerful technique that exploits optimal overlapping substructure in some problems.

1. Dynamic Programming

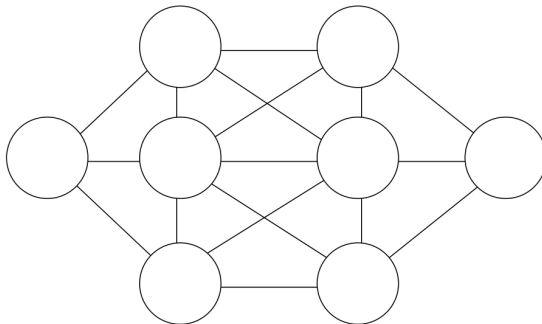
2. Constraint Programming

Number Circle Puzzle



Put a different number (1 to 8) in each circle such that adjacent circles do not have consecutive numbers.

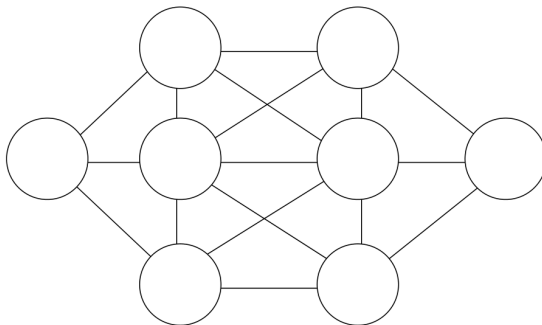
Number Circle Puzzle



You have 8 minutes

Put a different number (1 to 8) in each circle such that adjacent circles do not have consecutive numbers.

Number Circle Puzzle



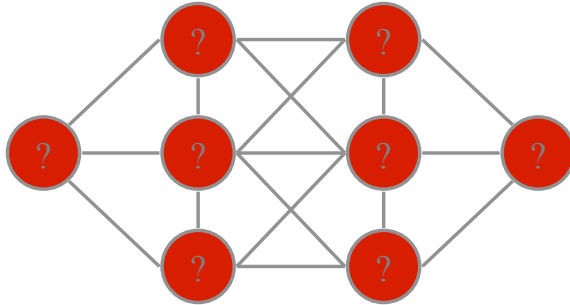
You have 8 minutes

Put a different number (1 to 8) in each circle such that adjacent circles do not have consecutive numbers.

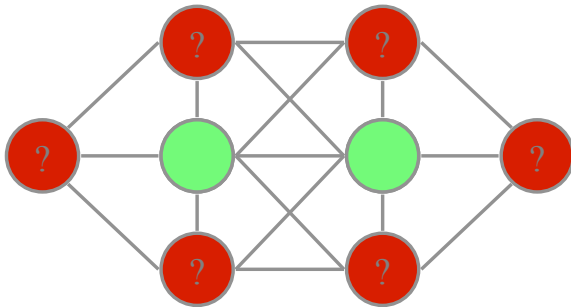
Example by Patrick Prosser with the help of Toby Walsh, Chris Beck, Barbara Smith, Peter van Beek, Edward Tsang, ...

Heuristic Search

Which nodes are hardest to number?

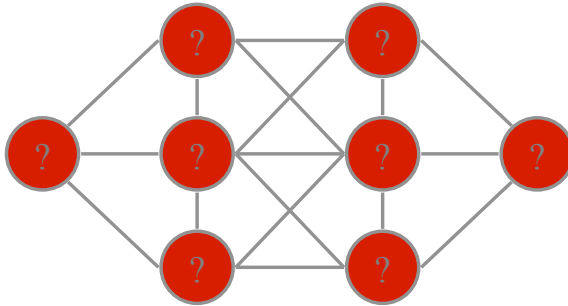


Heuristic Search



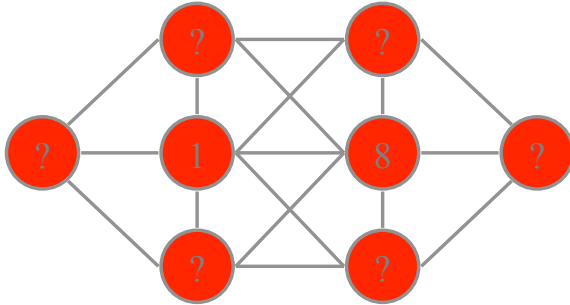
Heuristic Search

Which are the least constraining values to use?



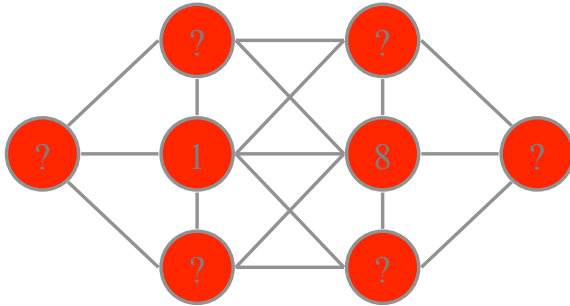
Heuristic Search

Values 1 and 8



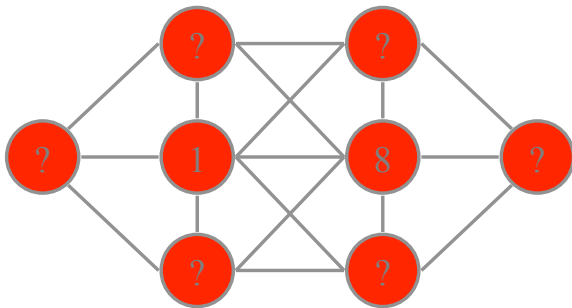
Heuristic Search

Values 1 and 8



Symmetry means we don't need to consider: 8 1

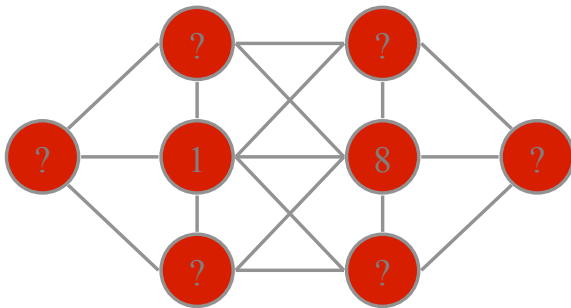
Inference/propagation



We can now eliminate many values for other nodes

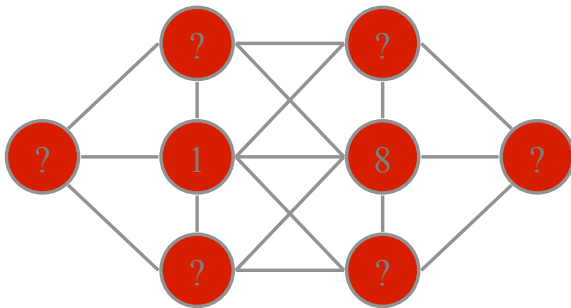
Inference/propagation

$\{1,2,3,4,5,6,7,8\}$



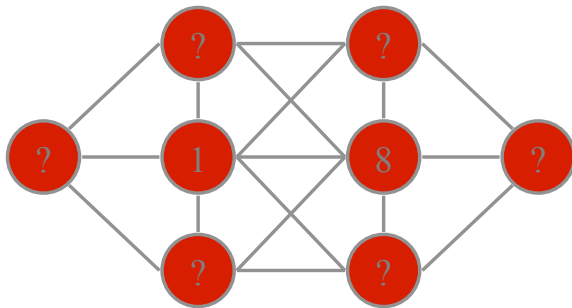
Inference/propagation

$\{2,3,4,5,6,7\}$

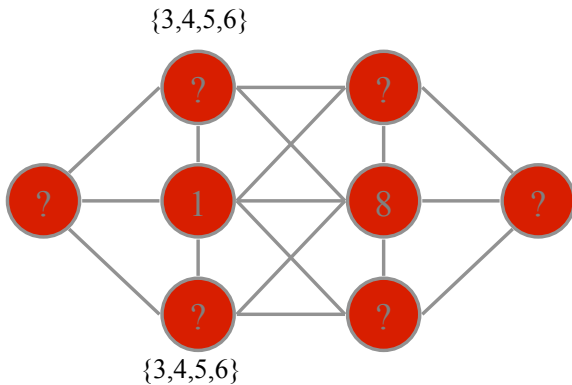


Inference/propagation

$\{3,4,5,6\}$

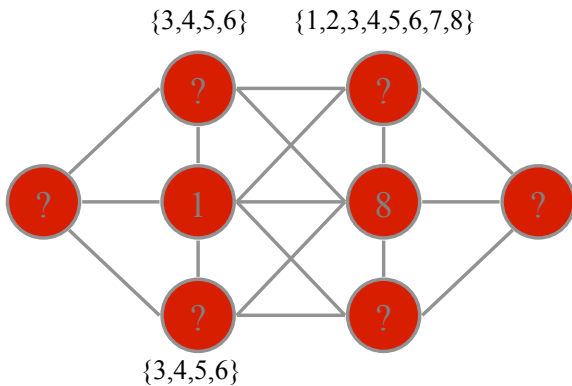


Inference/propagation

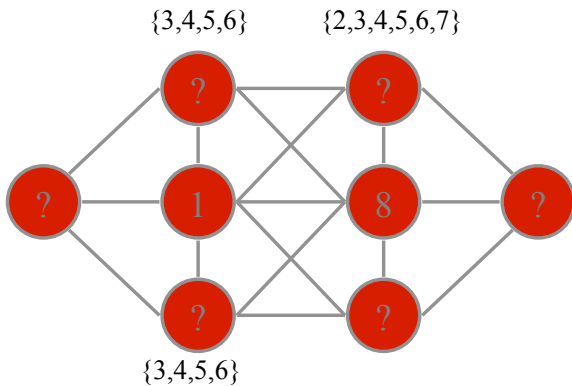


By symmetry

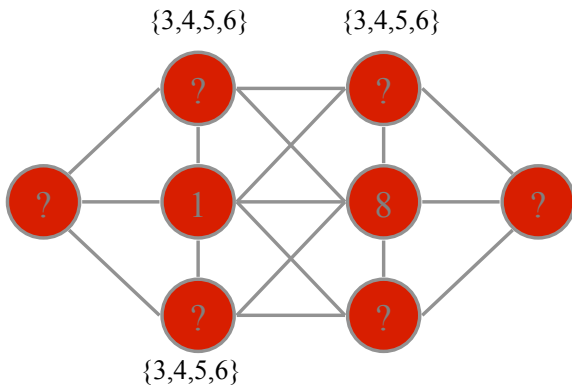
Inference/propagation



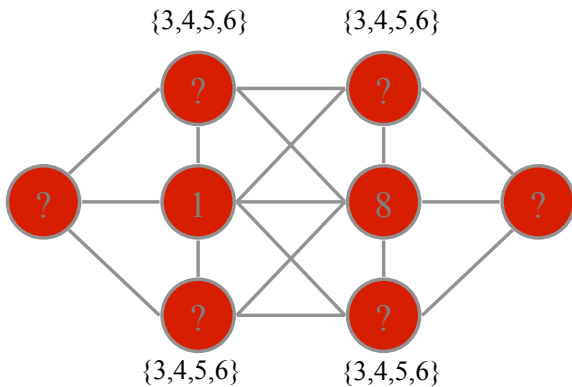
Inference/propagation



Inference/propagation

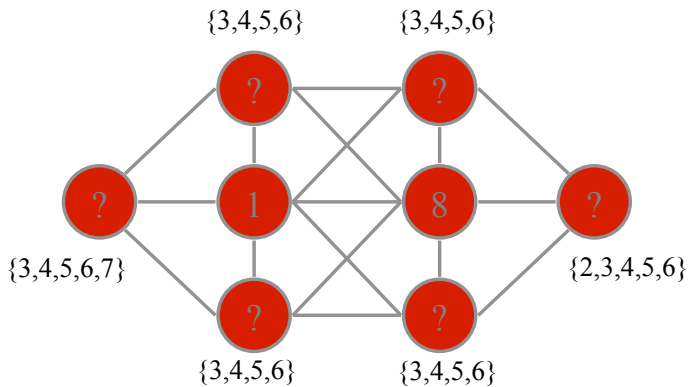


Inference/propagation

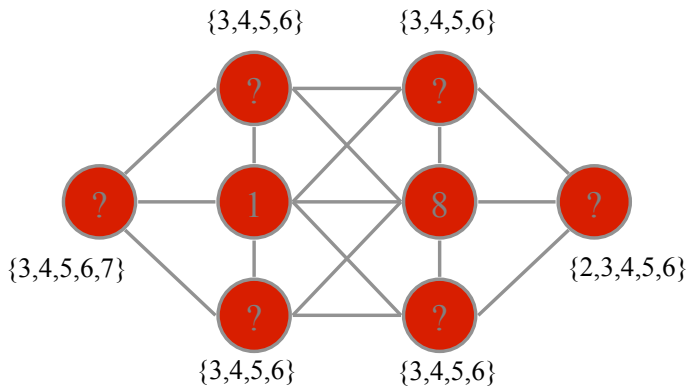


By symmetry

Inference/propagation

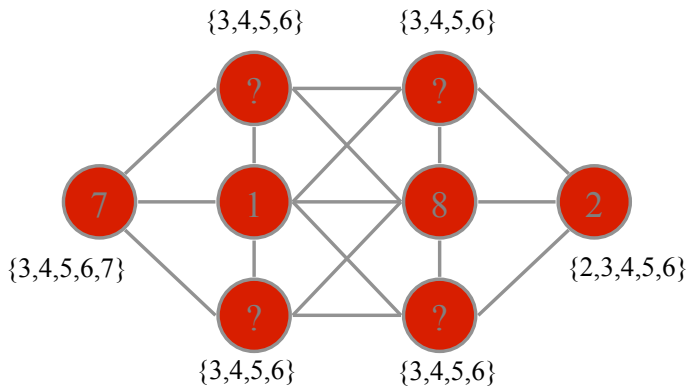


Inference/propagation



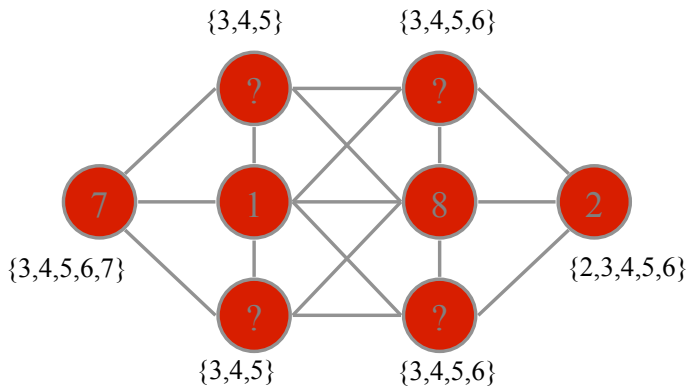
Value 2 and 7 are left in just one variable domain each

Inference/propagation



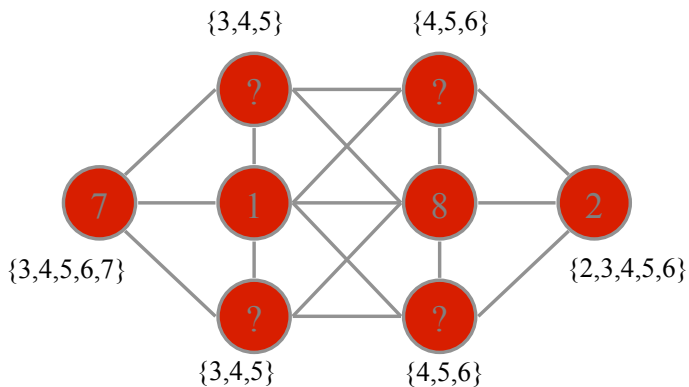
And propagate ...

Inference/propagation



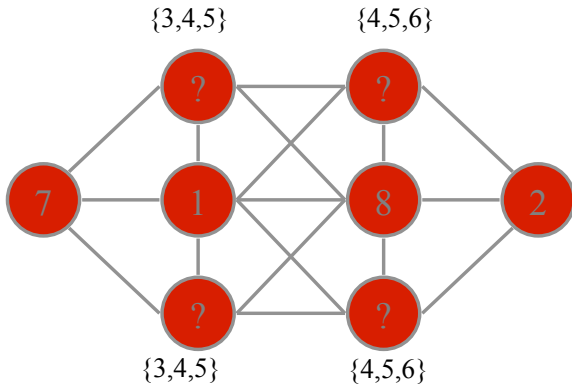
And propagate ...

Inference/propagation



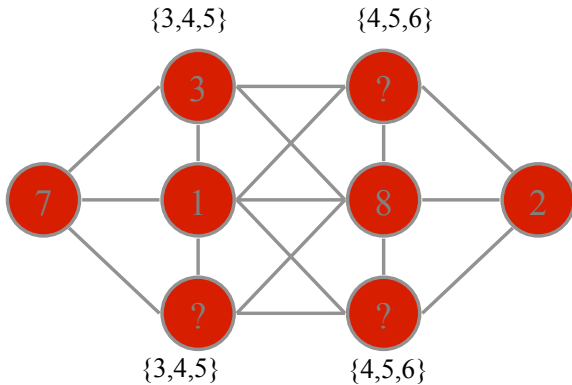
And propagate ...

Inference/propagation



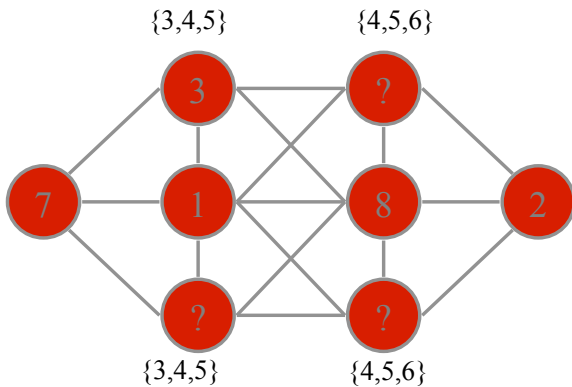
Guess a value, but be prepared to backtrack ...

Inference/propagation



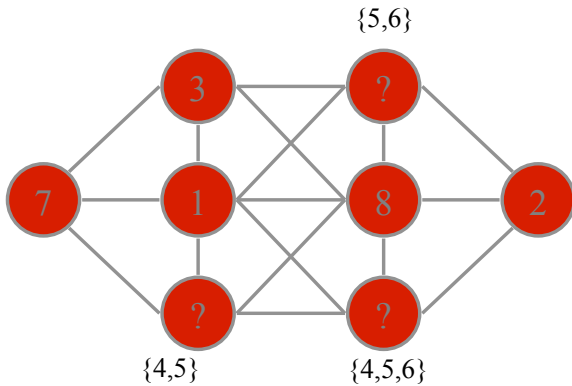
Guess a value, but be prepared to backtrack ...

Inference/propagation



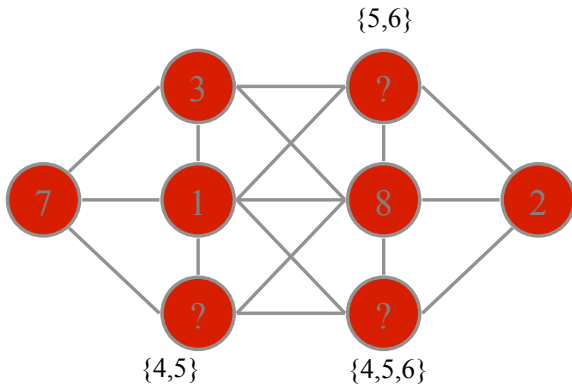
And propagate ...

Inference/propagation



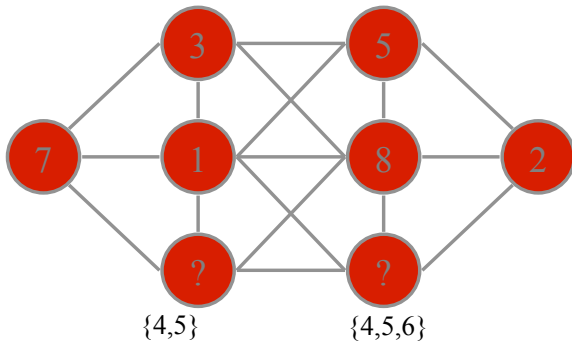
And propagate ...

Inference/propagation



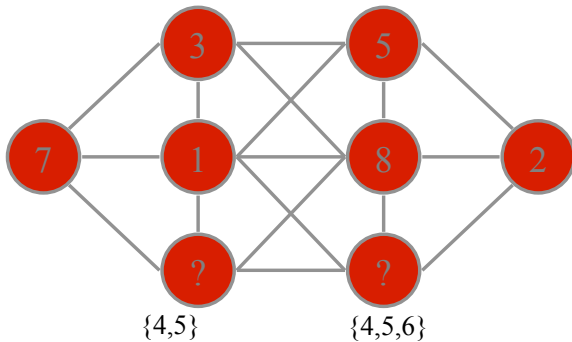
Guess another value ...

Inference/propagation



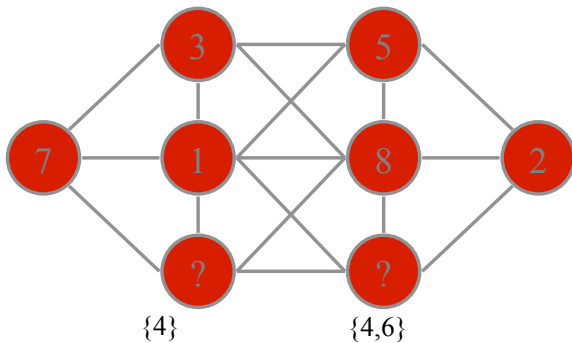
Guess another value ...

Inference/propagation



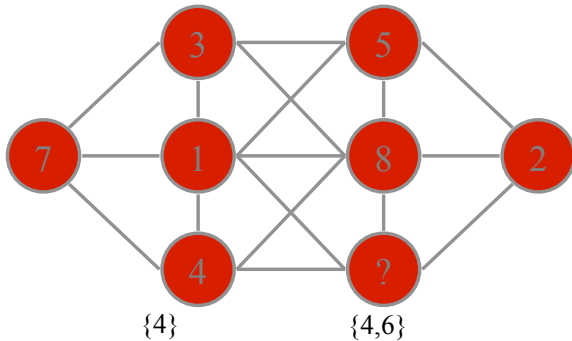
And propagate ...

Inference/propagation



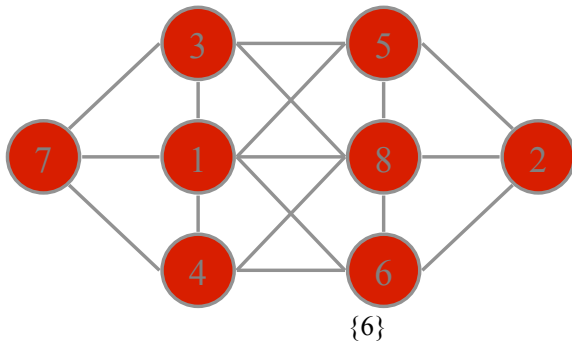
And propagate ...

Inference/propagation

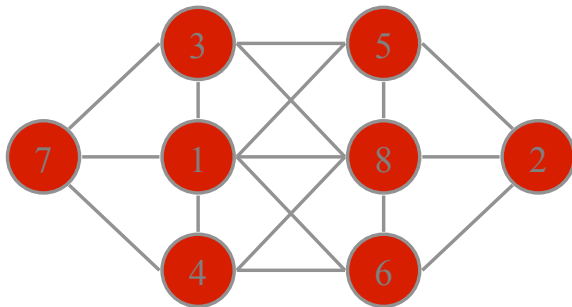


One node has only a single value left ...

Inference/propagation

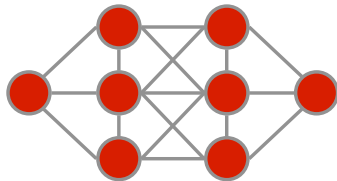


Solution



Hardness

- The puzzle is actually a hard problem
 - NP-complete



The Core of Constraint Computation

- Modelling
Deciding on variables/domains/constraints
- Inference/Propagation
- Heuristic Search
- Symmetry
- Backtracking

Problem-Solver Separation in Solution Paradigms

Dynamic Programming
Constraint Programming

1. Dedicated algorithms
(eg.: enumeration, branch and bound, dynamic programming)
2. Integer Linear Programming
3. Constraint Programming
4. SAT Solving and others
5. Randomized Search/Optimization Heuristics

Problem-Solver Separation in Solution Paradigms

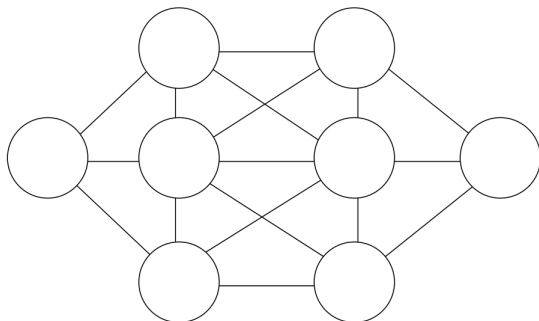
Dynamic Programming
Constraint Programming

1. Dedicated algorithms
(eg.: enumeration, branch and bound, dynamic programming)
2. Integer Linear Programming
3. Constraint Programming
4. SAT Solving and others
5. Randomized Search/Optimization Heuristics

Common to 2-5: **Representation (modeling) + reasoning (search + inference)**

- **Model** problem by specifying constraints on acceptable solutions:
 - define variables and domains
 - post constraints on these variables
- Solve model
 - choose algorithm
 - incremental assignments / backtracking search
 - complete assignments / (stochastic) local search
 - design heuristics

Constraint Satisfaction Problem



- Variable x_i for each node $i = 1, \dots, 8$
- Domain $\{1, \dots, 8\}$ for each variable x_i
- Constraints:

```
allDifferent([ $x_1, x_2, x_3, x_4, x_5, x_6, x_7, x_8$ ])  
 $|x_1 - x_2| > 1$   
 $|x_2 - x_3| > 1$   
 $|x_3 - x_4| > 1$   
 $\vdots$   
 $|x_7 - x_8| > 1$ 
```

Definition (Variables, Domains, Constraints)

The **domain** of a variable x , denoted $D(x)$, is a finite set of elements that can be assigned to x .

Definition (Variables, Domains, Constraints)

The **domain** of a variable x , denoted $D(x)$, is a finite set of elements that can be assigned to x .

A **constraint** C on X is a subset of the Cartesian product of the domains of the variables in X , i.e., $C \subseteq D(x_1) \times \cdots \times D(x_k)$. A tuple $(d_1, \dots, d_k) \in C$ is called a **solution** to C .

Definition (Variables, Domains, Constraints)

The **domain** of a variable x , denoted $D(x)$, is a finite set of elements that can be assigned to x .

A **constraint** C on X is a subset of the Cartesian product of the domains of the variables in X , i.e., $C \subseteq D(x_1) \times \cdots \times D(x_k)$. A tuple $(d_1, \dots, d_k) \in C$ is called a **solution** to C .

Equivalently, we say that a solution $(d_1, \dots, d_k) \in C$ is an assignment of the value d_i to the variable x_i for all $1 \leq i \leq k$, and that this assignment satisfies C .

Definition (Variables, Domains, Constraints)

The **domain** of a variable x , denoted $D(x)$, is a finite set of elements that can be assigned to x .

A **constraint** C on X is a subset of the Cartesian product of the domains of the variables in X , i.e., $C \subseteq D(x_1) \times \dots \times D(x_k)$. A tuple $(d_1, \dots, d_k) \in C$ is called a **solution** to C .

Equivalently, we say that a solution $(d_1, \dots, d_k) \in C$ is an assignment of the value d_i to the variable x_i for all $1 \leq i \leq k$, and that this assignment satisfies C .

If $C = \emptyset$, we say that it is **inconsistent**.

Extensional: specifies the good (or bad) tuples (values)

Intensional: specifies the characteristic function

Constraint Satisfaction Problem (CSP)

A CSP is a finite set of variables X with **domain extension** $\mathcal{D} = D(x_1) \times \cdots \times D(x_n)$, together with a finite set of constraints $\mathcal{C}$, each on a subset of X . A **solution** to a CSP is an assignment of a value $d \in D(x)$ to each $x \in X$, such that all constraints are satisfied simultaneously.

Constraint Satisfaction Problem (CSP)

A CSP is a finite set of variables X with **domain extension** $\mathcal{D} = D(x_1) \times \cdots \times D(x_n)$, together with a finite set of constraints $\mathcal{C}$, each on a subset of X . A **solution** to a CSP is an assignment of a value $d \in D(x)$ to each $x \in X$, such that all constraints are satisfied simultaneously.

Constraint Optimization Problem (COP)

A COP is a CSP $\mathcal{P}$ defined on the variables $x_1, \dots, x_n$, together with an objective function $f : D(x_1) \times \cdots \times D(x_n) \rightarrow Q$ that assigns a value to each assignment of values to the variables. An **optimal solution** to a minimization (maximization) COP is a solution d to $\mathcal{P}$ that minimizes (maximizes) the value of $f(d)$.

Partial vs **Complete** assignments

Variables can be combinatorial structures:

- integer variables (arrays)
- permutations/sequences
- sets with set domains
- ordered lists
- graphs

and constraints defined on them

Another Example: Social Golfers

Another Example: Social Golfers

Example (Social Golfer Problem (Combinatorial Design))

- 9 golfers: 1, 2, 3, 4, 5, 6, 7, 8, 9
- wish to play in groups of 3 players in 4 days
- such that no golfer plays in the same group with any other golfer more than just once.

Is it possible?

	Group 1	Group 2	Group 3
Day 0	???	???	???
Day 1	???	???	???
Day 2	???	???	???
Day 3	???	???	???

Another Example: Social Golfers

Example (Social Golfer Problem (Combinatorial Design))

- 9 golfers: 1, 2, 3, 4, 5, 6, 7, 8, 9
- wish to play in groups of 3 players in 4 days
- such that no golfer plays in the same group with any other golfer more than just once.

Is it possible?

	Group 1	Group 2	Group 3
Day 0	???	???	???
Day 1	???	???	???
Day 2	???	???	???
Day 3	???	???	???

This is an instance of a **constrained satisfaction problem**. Adding an optimizing criterion we get a **constrained optimization problem**.

Constraint Programming: Representation

Constraint Programming: Representation

Golfers

	Group 1	Group 2	Group 3
Day 0	? ? ?	? ? ?	? ? ?
Day 1	? ? ?	? ? ?	? ? ?
Day 2	? ? ?	? ? ?	? ? ?
Day 3	? ? ?	? ? ?	? ? ?

Constraint Programming: Representation

Golfers

	Group 1	Group 2	Group 3
Day 0	???	???	???
Day 1	???	???	???
Day 2	???	???	???
Day 3	???	???	???

Groups

	Day 0	Day 1	Day 2	Day 3
Golfer 0	1	{1,2,3}	{1,2,3}	{1,2,3}
Golfer 1	1	{1,2,3}	{1,2,3}	{1,2,3}
Golfer 2	1	{1,2,3}	{1,2,3}	{1,2,3}
Golfer 3	{2,3}	{1,2,3}	{1,2,3}	{1,2,3}
Golfer 4	{2,3}	{1,2,3}	{1,2,3}	{1,2,3}
Golfer 5	{2,3}	{1,2,3}	{1,2,3}	{1,2,3}
Golfer 6	{2,3}	{1,2,3}	{1,2,3}	{1,2,3}
Golfer 7	{2,3}	{1,2,3}	{1,2,3}	{1,2,3}
Golfer 8	{2,3}	{1,2,3}	{1,2,3}	{1,2,3}

Constraint Programming: Representation

Golfers

	Group 1	Group 2	Group 3
Day 0	???	???	???
Day 1	???	???	???
Day 2	???	???	???
Day 3	???	???	???

Integer variables:

`assign[i, j]` variables whose value is from domain $\{1, 2, 3\}$

Groups

	Day 0	Day 1	Day 2	Day 3
Golfer ₀	1	{1,2,3}	{1,2,3}	{1,2,3}
Golfer ₁	1	{1,2,3}	{1,2,3}	{1,2,3}
Golfer ₂	1	{1,2,3}	{1,2,3}	{1,2,3}
Golfer ₃	{2,3}	{1,2,3}	{1,2,3}	{1,2,3}
Golfer ₄	{2,3}	{1,2,3}	{1,2,3}	{1,2,3}
Golfer ₅	{2,3}	{1,2,3}	{1,2,3}	{1,2,3}
Golfer ₆	{2,3}	{1,2,3}	{1,2,3}	{1,2,3}
Golfer ₇	{2,3}	{1,2,3}	{1,2,3}	{1,2,3}
Golfer ₈	{2,3}	{1,2,3}	{1,2,3}	{1,2,3}

Golfers

	Group 1	Group 2	Group 3
Day 0	???	???	???
Day 1	???	???	???
Day 2	???	???	???
Day 3	???	???	???

Integer variables:

`assign[i, j]` variables whose value is from domain $\{1, 2, 3\}$

Constraints:

C1: each group has exactly groupSize players

C2: each pair of players only meets once

Groups

	Day 0	Day 1	Day 2	Day 3
Golfer ₀	1	{1,2,3}	{1,2,3}	{1,2,3}
Golfer ₁	1	{1,2,3}	{1,2,3}	{1,2,3}
Golfer ₂	1	{1,2,3}	{1,2,3}	{1,2,3}
Golfer ₃	{2,3}	{1,2,3}	{1,2,3}	{1,2,3}
Golfer ₄	{2,3}	{1,2,3}	{1,2,3}	{1,2,3}
Golfer ₅	{2,3}	{1,2,3}	{1,2,3}	{1,2,3}
Golfer ₆	{2,3}	{1,2,3}	{1,2,3}	{1,2,3}
Golfer ₇	{2,3}	{1,2,3}	{1,2,3}	{1,2,3}
Golfer ₈	{2,3}	{1,2,3}	{1,2,3}	{1,2,3}

In MiniZinc

```
int: golfers = 9;
int: groupSize = 3;
int: days = 4;
int: groups = golfers/groupSize;
set of int: Golfer = 1..golfers;
set of int: Day = 1..days;
set of int: Group = 1..groups;

array[Golfer, Day] of var Group: assign; % Variables

constraint
  % C1: Each group has exactly groupSize players
  forall (gr in Group, d in Day) ( % c1
    sum (g in Golfer) (bool2int(assign[g,d] = gr)) = groupSize
  ) /\
  % C2: Each pair of players only meets at most once
  forall (g1, g2 in Golfer, d1, d2 in Day where g1 != g2 /\ d1 != d2) (
    (bool2int(assign[g1,d1] = assign[g2,d1]) + bool2int(assign[g1,d2] = assign[g2,d2])) <= 1);

solve :: int_search([assign[i,j] | i in Golfer, j in Day ],
  first_fail, indomain_min, complete) satisfy;
```

Groups

Golfers

	Group 1	Group 2	Group 3
Day 0	0 1 2		
Day 1			
Day 2			
Day 3			

	Day 0	Day 1	Day 2	Day 3
Golfer 0	1	{1,2,3}	{1,2,3}	{1,2,3}
Golfer 1	1	{1,2,3}	{1,2,3}	{1,2,3}
Golfer 2	1	{1,2,3}	{1,2,3}	{1,2,3}
Golfer 3	{2,3}	{1,2,3}	{1,2,3}	{1,2,3}
Golfer 4	{2,3}	{1,2,3}	{1,2,3}	{1,2,3}
Golfer 5	{2,3}	{1,2,3}	{1,2,3}	{1,2,3}
Golfer 6	{2,3}	{1,2,3}	{1,2,3}	{1,2,3}
Golfer 7	{2,3}	{1,2,3}	{1,2,3}	{1,2,3}
Golfer 8	{2,3}	{1,2,3}	{1,2,3}	{1,2,3}

Groups

Golfers

	Group 1	Group 2	Group 3
Day 0	0 1 2	3 4 5	
Day 1			
Day 2			
Day 3			

	Day 0	Day 1	Day 2	Day 3
Golfer 0	1	{1,2,3}	{1,2,3}	{1,2,3}
Golfer 1	1	{1,2,3}	{1,2,3}	{1,2,3}
Golfer 2	1	{1,2,3}	{1,2,3}	{1,2,3}
Golfer 3	2	{1,2,3}	{1,2,3}	{1,2,3}
Golfer 4	2	{1,2,3}	{1,2,3}	{1,2,3}
Golfer 5	2	{1,2,3}	{1,2,3}	{1,2,3}
Golfer 6	{3}	{1,2,3}	{1,2,3}	{1,2,3}
Golfer 7	{3}	{1,2,3}	{1,2,3}	{1,2,3}
Golfer 8	{3}	{1,2,3}	{1,2,3}	{1,2,3}

Groups

Golfers

	Group 1	Group 2	Group 3
Day 0	0 1 2	3 4 5	6 7 8
Day 1			
Day 2			
Day 3			

	Day 0	Day 1	Day 2	Day 3
Golfer 0	1	{1,2,3}	{1,2,3}	{1,2,3}
Golfer 1	1	{1,2,3}	{1,2,3}	{1,2,3}
Golfer 2	1	{1,2,3}	{1,2,3}	{1,2,3}
Golfer 3	2	{1,2,3}	{1,2,3}	{1,2,3}
Golfer 4	2	{1,2,3}	{1,2,3}	{1,2,3}
Golfer 5	2	{1,2,3}	{1,2,3}	{1,2,3}
Golfer 6	3	{1,2,3}	{1,2,3}	{1,2,3}
Golfer 7	3	{1,2,3}	{1,2,3}	{1,2,3}
Golfer 8	3	{1,2,3}	{1,2,3}	{1,2,3}

Groups

Golfers

	Group 1	Group 2	Group 3
Day 0	0 1 2	3 4 5	6 7 8
Day 1	0		
Day 2			
Day 3			

	Day 0	Day 1	Day 2	Day 3
Golfer 0	1	1	{1,2,3}	{1,2,3}
Golfer 1	1	{2,3}	{1,2,3}	{1,2,3}
Golfer 2	1	{2,3}	{1,2,3}	{1,2,3}
Golfer 3	2	{1,2,3}	{1,2,3}	{1,2,3}
Golfer 4	2	{1,2,3}	{1,2,3}	{1,2,3}
Golfer 5	2	{1,2,3}	{1,2,3}	{1,2,3}
Golfer 6	3	{1,2,3}	{1,2,3}	{1,2,3}
Golfer 7	3	{1,2,3}	{1,2,3}	{1,2,3}
Golfer 8	3	{1,2,3}	{1,2,3}	{1,2,3}

Groups

Golfers

	Group 1	Group 2	Group 3
Day 0	0 1 2	3 4 5	6 7 8
Day 1	0	1	
Day 2			
Day 3			

	Day 0	Day 1	Day 2	Day 3
Golfer 0	1	1	{1,2,3}	{1,2,3}
Golfer 1	1	2	{1,2,3}	{1,2,3}
Golfer 2	1	{3}	{1,2,3}	{1,2,3}
Golfer 3	2	{1,2,3}	{1,2,3}	{1,2,3}
Golfer 4	2	{1,2,3}	{1,2,3}	{1,2,3}
Golfer 5	2	{1,2,3}	{1,2,3}	{1,2,3}
Golfer 6	3	{1,2,3}	{1,2,3}	{1,2,3}
Golfer 7	3	{1,2,3}	{1,2,3}	{1,2,3}
Golfer 8	3	{1,2,3}	{1,2,3}	{1,2,3}

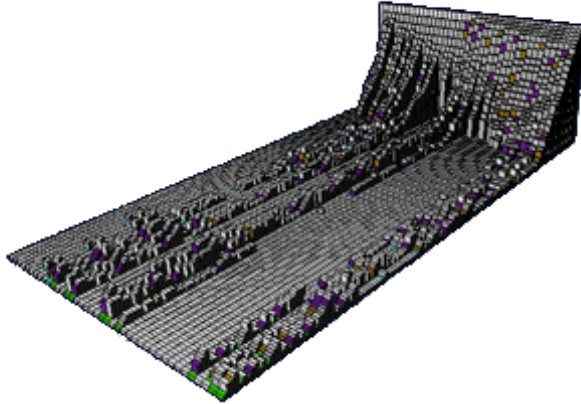
Groups

Golfers

	Group 1	Group 2	Group 3
Day 0	0 1 2	3 4 5	6 7 8
Day 1	0 3 6	1 4 7	2 5 8
Day 2	0 4 8	1 5 6	2 3 7
Day 3	0 5 7	1 3 8	2 4 6

	Day 0	Day 1	Day 2	Day 3
Golfer 0	1	1		
Golfer 1	1	2		
Golfer 2	1	{3}		
Golfer 3	2			
Golfer 4	2			
Golfer 5	2			
Golfer 6	3			
Golfer 7	3			
Golfer 8	3			

Constraint Programming: Reasoning



The solution process proceeds by propagating the constraints on the domains of the variables (ie, removing values) and tentatively assigning variables until only feasible values are left or backtracking.

- Two different approaches to solve combinatorial optimization problems: Dynamic programming and Constraint programming
- Dynamic programming is a dedicated approach that exploits optimal overlapping substructure in some problems
- Constraint programming is a general purpose approach that exploits the structure of (standard) constraints to reduce the search space